02 / ANÁLISIS DE LA INYECCIÓN

Cómo se alteraba la lógica del login

Reconstrucción de la consulta y efecto de los parámetros en SQLite

Angel Perez Reyes | 26 de octubre de 2026

RESUMEN

Este análisis sigue el recorrido de una entrada maliciosa desde el formulario hasta SQLite. La falla no estaba en la interfaz: aparecía cuando el servidor convertía datos externos en parte de la instrucción SQL. Al separar la consulta y sus valores, el mismo texto dejó de funcionar como operador.

01 Punto de entrada

El navegador envía username y password como JSON al endpoint POST /login-vulnerable. El servidor busca una fila de la tabla users; si la encuentra, responde con ok: true. En la versión inicial, los datos del cliente se insertaban entre comillas dentro del SELECT.

02 Secuencia del ataque

Una prueba de la práctica usa como usuario ' OR 1=1 -- y como contraseña cualquier texto. La primera comilla termina el valor vacío de username; OR 1=1 hace verdadera la condición; -- convierte el resto de la línea en comentario, incluida la comparación de la contraseña.

Como la consulta puede devolver varias filas, db.get toma la primera. En la demostración, esa fila corresponde a admin. Por eso el formulario mostraba ok: true aunque no se conociera su contraseña.

CONSULTA ORIGINAL / REPRESENTACIÓN DEL FALLO

```
SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = 'x'
```

03 Por qué cambia el resultado

La consulta parametrizada mantiene fijo el SELECT antes de incorporar los valores. SQLite compara el texto completo ' OR 1=1 -- con el campo username. No interpreta la comilla, el OR ni el comentario como sintaxis SQL.

No se resolvió el problema mediante una lista de palabras prohibidas. Filtrar OR, comillas o comentarios sería frágil y podría rechazar entradas legítimas. La separación entre código y datos actúa sobre la causa.

04 Contraste observado

La versión vulnerable está documentada en el reporte de referencia y se confirmó en la sesión de práctica antes de la corrección. Después del cambio, estas respuestas se comprobaron contra el servidor local:

CONSULTA PARAMETRIZADA / COMPARACIÓN

```
SELECT * FROM users WHERE username = ? AND password = ?
Valores: ["' OR 1=1 -- ", "x"]
```

Caso	Resultado tras el cambio
admin / admin	200 - acceso correcto
' OR 1=1 -- / x	401 - sin acceso
admin'-- / z	401 - sin acceso
' OR '1'='1 / igual	401 - sin acceso

Conclusión

El ataque dependía de que el servidor permitiera a un valor cerrar un literal y modificar el WHERE. Los parámetros impiden esa transición: el SQL conserva su significado y los intentos de inyección se convierten en simples nombres de usuario que no existen.

Fuente: reporte de práctica proporcionado por el usuario y verificación local del proyecto SeguridadInformatica (login/index.html y login/server-vulnerable.js).

EVIDENCIA VISUAL / ANGEL PEREZ REYES

Capturas de la práctica

Las dos capturas muestran el mismo intento de inyección antes y después de parametrizar la consulta.

01 Antes de la corrección



El texto de inyección con contraseña «x» obtuvo ok: true y devolvió admin.

02 Después de la corrección



La consulta parametrizada rechazó el mismo intento: ok: false y «Credenciales inválidas».

Capturas del navegador proporcionadas por el usuario durante la práctica.